

Legal RAG System

Project Report: Approach and Implementation

July 15, 2026

1 Project Overview

This project implements a Retrieval-Augmented Generation (RAG) system for the Legal domain. The system allows legal professionals to ask natural-language questions over a corpus of legal documents and receive concise, context-grounded answers accompanied by document names and page-level references, enabling verification against primary sources.

The end-to-end pipeline is:

Legal Professional → Streamlit UI / FastAPI API → Hybrid Retrieval (Elasticsearch) → CrossEncoder Reranking → Groq LLM → Legal Answer + Document Name + Page References

The workflow proceeds as follows: a user submits a question; the system performs hybrid (keyword + semantic) search over the indexed corpus; results are reranked for relevance; the top passages are supplied as context to an LLM; the LLM generates a grounded answer; and the system returns the answer together with source document names, types, and page ranges.

Two access methods are provided:

- A **FastAPI** backend exposing REST endpoints (e.g., `POST /ask`).
- A **Streamlit** web interface for conversational legal research.

Note on scope: the assignment specifies 100 documents; the repository contains 107 raw PDFs, of which 106 were successfully processed (one PDF, `IF12621.8.pdf`, is absent from the ingestion output).

2 Business Problem and Motivation

Legal text is dense, highly structured, and sensitive to exact wording. Pure keyword search can miss passages that discuss the same concept in different words; pure semantic search can miss exact statutory citations, case names, or tax form numbers. This project addresses the tension through a **hybrid retrieval** strategy:

- **Keyword search** retrieves exact terms such as “IRC Section 162,” “26 U.S.C.,” case names, tax forms, and legal phrases.
- **Vector search** retrieves semantically similar passages even when wording differs.
- **Reranking** prioritizes the most relevant passages before they reach the LLM.
- **Page-level metadata** is preserved end-to-end so answers remain verifiable.

The design goal is not simply to generate an answer, but to generate an answer that a lawyer can check against the original source document and page.

3 Milestone 1 — Data Ingestion and Pre-processing

3.1 Objective

Convert raw legal PDFs into clean, searchable, citation-ready data while preserving page numbers.

3.2 Dataset

Category	Meaning	Raw PDF Count
Acts	Statutory laws and public laws	29
Court Judgments	Court opinions and case law	38
POV	Legal commentary / analysis	31
Tax Documents	IRS, CFR, tax material	9
Total		107

The processed dataset contains 106 successfully represented documents, 5,241 extracted pages, 7,381 document chunks, and 7,381 embeddings.

3.3 PDF Parsing

Implemented in `ingestion/pdf_parser.py` using PyMuPDF (`fitz`) for native text extraction, Tesseract OCR (`pytesseract`) for image-based/scanned pages, Pillow for rendering page images prior to OCR, and UUIDs for unique document/page identifiers. Each page record retains document ID, name, type, source file, page number, page ID, character/word counts, and extracted text — preserving the document–page relationship for later citation.

OCR handling: the parser extracts machine-readable text first; if a page yields no text, it is rendered at 300 DPI and passed to Tesseract.

Identified limitations:

- `OCR.TEXT.THRESHOLD = 25` is defined but unused — OCR triggers only on completely blank pages, so sparse or corrupted text may not receive OCR.
- The Tesseract executable path is hardcoded (`D:\Tesseract-OCR\tesseract.exe`) and should be moved to an environment variable for deployment portability.

3.4 Text Cleaning

Implemented in `ingestion/cleaner.py`. The cleaner removes excessive whitespace, normalizes repeated newlines, fixes hyphenation breaks across lines (e.g., “busi-ness” → “business”), converts smart quotes to standard quotes, and strips invisible control characters while preserving substantive legal language. Output is saved to `data/processed/clean_pages.json`.

3.5 Document Chunking

Implemented in `ingestion/chunker.py`. Pages of a document are combined in order before chunking, since legal provisions often span page boundaries.

Setting	Value
Chunk size	400 words
Chunk overlap	80 words
Minimum trailing chunk size	50 words

Each chunk stores chunk/document IDs, type, chunk index, page start/end, page IDs, word/character counts, and text — forming the basis of page-level citations.

3.6 Metadata Extraction

Implemented in `ingestion/metadata_extractor.py` using rule-based and regex methods to extract US Code and CFR citations, statutory sections, act/case/court/judge names, tax forms and IRS publications, tax years, dates, titles/chapters/articles, keywords (via corpus-aware IDF-style scoring), and named entities.

Metadata Field	Chunks Containing Field
Keywords	7,357
Sections	3,687
Entities	3,510
Dates	2,916
US Codes	citations
947	
Tax forms	214
Titles	224
IRS publications	49
CFR citations	38

4 Milestone 2 — Search Architecture and Indexing

4.1 Objective

Implement hybrid search over Elasticsearch to retrieve both exact legal terms/citations and semantically relevant passages, using Elasticsearch as both a BM25 keyword engine and a dense-vector database.

4.2 Embedding Generation

Implemented in `indexing/embedding_generator.py` and `indexing/embedding_pipeline.py` using **BAAI/bge-small-en-v1.5**, producing 384-dimensional embeddings. Queries are prefixed with BGE’s recommended instruction (“Represent this sentence for searching relevant passages:”) to improve query-to-passage retrieval.

4.3 Elasticsearch Index

Created by `indexing/create_index.py` as index `legal_rag_chunks`, with fields including `chunk_id`, `document_id`, `document_name`, `document_type`, `page_start`/`page_end`, `text`, `metadata`, and a `dense_vector` embedding field (cosine similarity, indexed for KNN). Bulk indexing (`indexing/index_documents.py`) uses a batch size of 500 chunks.

4.4 Keyword Search

Implemented in `retrieval/keyword_search.py` using BM25, with field boosts: `document_name`, `sections`, `usc_citations`, `cfr_citations` (high); `acts`, `cases`, `keywords` (medium); `document_type` (mild); `text` (base). This ensures queries like “26 U.S.C. 162” strongly favor exact-citation chunks.

4.5 Vector Search

Implemented in `retrieval/vector_search.py`: the question is embedded via BGE, sent to Elastic-search, and matched via KNN against at least 200 candidates before returning top results — balancing recall and latency.

4.6 Hybrid Search with Reciprocal Rank Fusion (RRF)

Implemented in `retrieval/hybrid_search.py`. BM25 and vector result lists are combined via RRF ($k = 60$):

$$RRFScore = \sum \frac{1}{k + rank}$$

RRF is preferred over direct score averaging since BM25 and vector scores are on incompatible scales. The default flow retrieves candidates from both methods, fuses via RRF, and returns the top 20 fused chunks. Filters for document type, allowed document IDs, and allowed chunk IDs are supported (intended for future GraphRAG integration).

4.7 CrossEncoder Reranking

Implemented in `retrieval/reranker.py` using **cross-encoder/ms-marco-MiniLM-L-6-v2**, which jointly scores (question, chunk) pairs for improved precision over independently-encoded vector similarity.

Stage	Number of Results
Hybrid candidate pool	50
Final reranked chunks	4

Final flow: *Query* → BM25 + Vector Search → RRF Fusion → Top 50 → CrossEncoder Rerank → Top 4 chunks to LLM.

4.8 GraphRAG and OKF Status

The assignment calls for relationship mapping (e.g., judgment → Act citations), Neo4j/GraphRAG usage, and OKF standardization. The repository includes early groundwork only: Neo4j environment variables in `config.py`, graph-related folders/paths, metadata fields that could become graph nodes/edges, and retrieval filters designed for graph-derived allowlists. **No active code** connects to Neo4j, builds graph nodes/relationships, extracts citation edges, queries graph relationships pre-retrieval, or produces a confirmed OKF structure. GraphRAG and OKF are therefore **planned / partially scaffolded infrastructure**, not completed functionality.

5 Milestone 3 — Feature Development and User Experience

5.1 Question-and-Answer Interface

A. FastAPI Backend (`api/main.py`):

Endpoint	Method	Purpose
/	GET	Confirms API is running
/health	GET	Returns health status
/stats	GET	Returns corpus statistics
/ask	POST	Submits a legal question

Example response includes the generated answer, latency, model name, and a references list of document name, type, and page start/end.

B. Streamlit Web Interface (`app.py`, intended for Hugging Face Spaces): conversational input, suggested questions, response time and model display, reference display, copy/download (`.md/.txt`) of answers, conversation clearing, and corpus statistics in the sidebar. `ui/app_with_backend.py` is an alternate client that calls the FastAPI backend over HTTP.

5.2 Summarization

Orchestrated by `rag/rag_pipeline.py`: validate the question, retrieve and rerank chunks, build a legal-context prompt (`rag/prompt_builder.py`), call the Groq LLM, and return the answer with model details, token usage, latency, and references. The system prompt instructs the model to use only supplied context, avoid inventing statutes/cases, state when context is insufficient, use professional legal language, include a References section, and follow source precedence: Acts → Court Judgments → Tax Documents → POV Documents. Context is capped at 40,000 characters.

5.3 LLM and Groq Integration

Implemented in `rag/groq_client.py` with fallback models: `openai/gpt-oss-120b`, `llama-3.3-70b-versatile`, `qwen/qwen3-32b`, `meta-llama/llama-4-scout-17b-16e-instruct`, `openai/gpt-oss-20b`. The client reads the Groq API key from `.env`, tracks token usage and latency, uses deterministic generation (`temperature = 0`), retries with an alternate model on transient failures (rate limits, timeouts, server errors), and stops immediately on permanent errors (invalid key, malformed request).

5.4 Citation and Verification Features

Page references are preserved from PDF parsing through final generation, with each chunk carrying document name, type, and start/end pages displayed in the UI. Provenance is maintained across the full pipeline: PDF → text → chunks → metadata → index → retrieval → rerank → API → UI.

Current limitation: returned references are the chunks *supplied* to the model, not a verified list of citations the LLM actually used in its prose — so citation extraction/validation is needed for production use.

6 Milestone 4 — Quality Assurance and Evaluation

6.1 Golden Set

Stored at `data/golden_set/golden_set.csv` with Question and Ground Truth Answer fields, containing 199 questions. `evaluation/generate_llm_answers.py` runs the pipeline per question, saves results to `evaluation/llm_gen_csv.csv`, and supports partial continuation. Currently **147 of 199** questions have been processed.

6.2 LLM-as-a-Judge Evaluation

`evaluation/evaluate_answers.py` uses a Groq model to compare ground truth vs. generated answers, producing a **Document Retrieval Score** and a **Faithfulness Score**, saved to `evaluation/evaluation_report`

Metric	Score
Reference Retrieval Accuracy	54.42%
Faithfulness	67.35%
Overall Reliability	46.94%
Hallucination Rate	32.65%
Citation Error Rate	45.58%

6.3 Interpretation of Results

The prototype is technically functional, but legal answer quality needs improvement. A 67.35% faithfulness score indicates answers are frequently aligned with ground truth. However, reference retrieval accuracy (54.42%) and citation error rate (45.58%) indicate that citation accuracy — critical for legal research — is a primary weakness. **Conclusion:** the system is a functional prototype suited to legal-research assistance where a lawyer verifies all output against primary sources, not for unsupervised legal advice.

6.4 Why the Scores Are Not Higher

A likely root cause of the moderate scores is the small final context size: only the **top $k = 4$** reranked chunks are sent to the LLM per query. This was a deliberate constraint driven by the **tokens-per-minute (TPM) rate limit of the free Groq API tier**, not a modeling choice — sending more chunks per request increases the risk of hitting rate limits and failing requests, both during live use and during batch Golden Set evaluation. With a higher TPM allowance (a paid Groq tier, or an alternative provider), **k** could be increased to supply more supporting context per question, which would likely improve both reference retrieval accuracy and faithfulness, since the correct passage is more likely to be present in a larger context window.

The evaluation was also run under the same time and resource constraints as development: only 147 of 199 Golden Set questions were completed, and there was no time to systematically test alternative LLMs or retrieval configurations against the Golden Set before this report was written. The reported metrics should therefore be read as a lower bound on the architecture’s achievable performance rather than a ceiling.

7 Technology Stack

Layer	Technology
Programming language	Python
PDF extraction	PyMuPDF
OCR	Tesseract + Pillow
Data format	JSON / CSV
Embeddings	Sentence Transformers
Embedding model	BAAI/bge-small-en-v1.5
Vector search	Elasticsearch KNN
Keyword search	Elasticsearch BM25
Hybrid retrieval	Reciprocal Rank Fusion
Reranking	CrossEncoder
Reranker model	ms-marco-MiniLM-L-6-v2
LLM provider	Groq
Backend API	FastAPI
Frontend	Streamlit
Validation	Pydantic
Evaluation	Pandas + Groq judge
Future graph layer	Neo4j configuration

8 Deployment Notes

The project is deployed with the following resources:

- **Deployed app:** <https://legal-rag-search-ss04.streamlit.app>
- **Deployed app repository:** <https://github.com/ShyamSuman-04/legal-rag-search>
- **Main project repository:** <https://github.com/ShyamSuman-04/legal-rag-system>

To fit within the memory constraints of Streamlit Community Cloud’s free tier, two changes were made in the deployed version relative to the main repository:

- The CrossEncoder reranking stage (`reranker.py`) was removed.
- The embedding model was switched from BAAI/bge-small-en-v1.5 to the lighter sentence-transformer reducing RAM usage during deployment.

The main entry point for the deployed Streamlit app is `streamlit.py` in the deployed app repository.

9 Important Limitations and Improvements

1. **Retrieval confidence threshold:** the system currently returns the best available chunks even when weakly related; a confidence threshold should trigger an “insufficient context” response to reduce hallucination risk.
2. **Citation validation:** extract cited document names/pages from the LLM output, compare against actual retrieved chunk metadata, and flag or remove unsupported citations.
3. **System/user/context message separation:** splitting the current single assembled prompt into distinct system, user, and context messages would improve instruction hierarchy and prompt-injection resistance.
4. **Complete Golden Set evaluation:** 52 of 199 questions remain unprocessed.
5. **Automated tests:** none currently exist; recommended coverage includes page metadata preservation, chunk page-range accuracy, citation extraction, Elasticsearch query construction, RRF fusion behavior.

ior, reranking order, empty-query validation, prompt truncation, API schema validation, and citation validation.

6. **GraphRAG implementation:** a full extension would model documents citing Acts, referencing US Code sections, being decided by Courts, authored by Judges, and discussing legal topics — enabling queries such as “Which judgments cite a particular Act?” Graph results could pre-filter Elasticsearch retrieval before hybrid search runs.
7. **Increase the context window (k) sent to the LLM:** the current $k = 4$ limit exists because of the Groq free-tier tokens-per-minute cap. Moving to a paid tier, or a provider with a higher TPM allowance, would allow more reranked chunks to be included per prompt, which should directly improve reference retrieval accuracy and reduce the citation error rate.
8. **Migrate embeddings and vector search to Pinecone:** due to time constraints, the current pipeline generates and stores embeddings locally and runs vector/hybrid search on the same Elasticsearch instance as keyword search. Moving embedding generation and storage to **Pinecone**, and using its managed **dense-sparse hybrid search**, would offload this work to the cloud, reduce local compute/RAM load (relevant for free-tier deployment), and potentially simplify scaling the corpus beyond its current size.
9. **Benchmark alternative LLMs:** only a small, fixed set of Groq-hosted fallback models was used, chosen mainly for availability rather than measured task performance. Future work should systematically test additional LLMs (both larger Groq-hosted models and other providers) against the Golden Set to identify the model that yields the best faithfulness and citation accuracy for this legal-domain task, rather than relying on a single default choice.

10 Deliverables Mapping

Assignment Deliverable	Project Implementation
Architecture diagram	Included in this report
PDF ingestion	<code>ingestion/pdf_parser.py</code>
Page-index preservation	Page metadata and chunk page ranges
Cleaning	<code>ingestion/cleaner.py</code>
Chunking	<code>ingestion/chunker.py</code>
Metadata extraction	<code>ingestion/metadata_extractor.py</code>
Vector search	Elasticsearch KNN + BGE embeddings
Keyword search	Elasticsearch BM25
Hybrid search	RRF fusion
Reranking	CrossEncoder
Q&A system	<code>rag/rag_pipeline.py</code>
API demo	FastAPI /ask endpoint
UI demo	Streamlit app
Golden Set	<code>data/golden_set/golden_set.csv</code>
Evaluation	Evaluation scripts and CSV report
GraphRAG	Planned / partially scaffolded, not implemented
OKF	Planned / configured conceptually, not implemented